

Hospital Management System v-2 Project Report

1. Student Details

- **Name:** Rehan Alam
 - **Roll Number:** 23f2002420
 - **Email:** 23f2002420@ds.study.iitm.ac.in
 - **About Me:** I am a student in the IIT Madras BS Degree program with a strong interest in full-stack web development, Machine Learning , Deep Learning and GenAI.
-

2. Project Details

- **Project Title:** Hospital Management System (HMS)
 - **Problem Statement:** To design and build a modern, role-based web application to replace traditional, inefficient hospital management methods. The system solves problems like scheduling conflicts, disconnected patient records, and the difficulty in tracking doctor availability by providing a single, centralized platform for all hospital operations.
 - **Approach:** The application is built with a decoupled architecture. The backend is a robust Flask API responsible for all business logic, database operations, and secure authentication. The frontend is a responsive Single Page Application (SPA) built in Vue.js, which consumes the Flask API. This approach allows for a clean separation of concerns and a fast, modern user experience for the three distinct roles: Admin, Doctor, and Patient.
-

3. AI/LLM Declaration

- I used Google Gemini as a programming assistant, primarily for debugging and refactoring my code.
- I wrote the core application logic and features myself.
- When I encountered an error (e.g., a Python traceback in my Flask terminal or a console error in Vue), I provided the error message to Gemini.
- Gemini helped me analyze the error and find the specific line of code that had the problem (such as typos, logic errors, or incorrect configurations).
- I also used it to get suggestions on how to refactor my existing, duplicated code into cleaner, reusable versions.
- All final implementation logic, debugging, API integration, and project structure were done manually by me, Overall 10-12% .

4. Technologies and Frameworks Used

Technology / Library Purpose

Flask	Core backend web framework for building the REST API.
Vue.js	Frontend JavaScript framework for building the responsive UI.
SQLAlchemy	Object Relational Mapper (ORM) for database interactions.
Bootstrap	Frontend CSS framework for styling and responsive design.
Celery	Asynchronous task queue for handling background jobs.
Redis	In-memory data store, used as the message broker for Celery.
Flask-JWT-Extended	For secure user authentication and role-based access control.

5. Database Schema

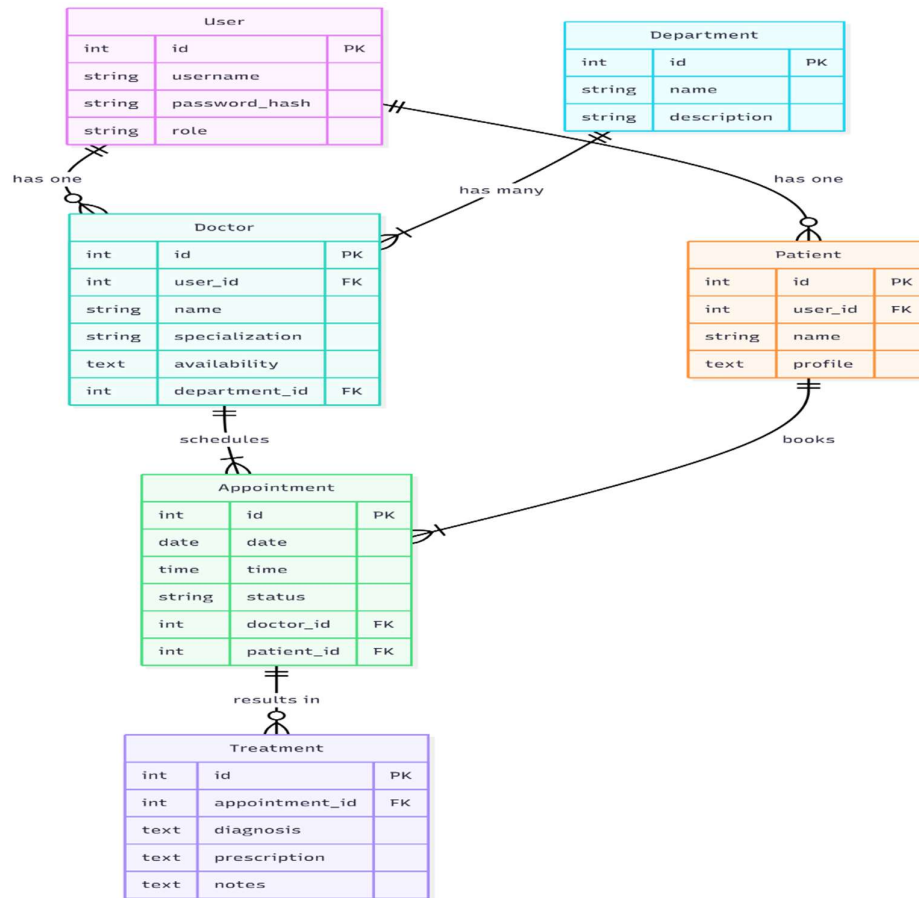
Tables:

- **User:** Stores login details for all roles (id, username, password_hash, role).
- **Doctor:** Stores doctor-specific profiles (id, user_id, name, specialization, department_id, availability).
- **Patient:** Stores patient-specific profiles (id, user_id, name, profile).
- **Department:** Stores hospital departments (id, name, description).
- **Appointment:** Logs all appointment details (id, date, time, status, doctor_id, patient_id).
- **Treatment:** Stores details of a completed appointment (id, appointment_id, diagnosis, prescription, notes).

Relationships:

- One-to-One: User → Doctor
- One-to-One: User → Patient
- One-to-Many: Department → Doctor
- One-to-Many: Doctor → Appointment

- One-to-Many: Patient → Appointment
- One-to-One: Appointment → Treatment



6. API Resource Endpoints

A selection of key API endpoints implemented:

Endpoint	Method	Description
/api/login	POST	Authenticates any user (Admin, Doctor, Patient) and issues a JWT.
/api/register	POST	Creates a new Patient account.
/api/admin/dashboard	GET	Fetches dashboard statistics for the Admin.

Endpoint	Method	Description
/api/admin/doctors	POST	(Admin) Creates a new doctor profile and user account.
/api/admin/departments	GET/POST	(Admin) Fetches or creates a new department.
/api/admin/departments/<id>	PUT/DELETE	(Admin) Updates or deletes a specific department.
/api/doctor/appointments	GET	(Doctor) Fetches all appointments assigned to the logged-in doctor.
/api/doctor/availability	PUT	(Doctor) Updates the doctor's weekly availability schedule.
/api/appointments/<id>/complete	PUT	(Doctor) Marks an appointment as complete and saves treatment notes.
/api/patient/appointments	GET	(Patient) Fetches all appointments for the logged-in patient.
/api/appointments	POST	(Patient) Books a new appointment.
/api/appointments/<id>/cancel	PUT	(Patient) Cancels a booked appointment.
/api/patient/export-history	POST	(Patient) Triggers a Celery task to email a CSV of their history.

7. Architecture and Features

Architecture Overview:

- **/backend:** The Flask application root.
 - app.py: Main application entry point, creates Flask app, initializes Celery.
 - /application/models.py: All SQLAlchemy database models.
 - /application/routes.py: All Flask API endpoints and logic.
 - celery_config.py: Celery broker/backend and Beat schedule definitions.

- celery_init.py: Celery-Flask integration factory.
 - tasks.py: All Celery background task definitions.
- **/frontend:** The Vue.js project.
 - /src/components: All reusable Vue components (dashboards, pages, navbar, etc.).
 - /src/routes.js: Vue Router definitions for all frontend pages.

Implemented Features:

- Full role-based authentication for Admin, Doctor, and Patient.
- **Patient:** Can register, log in, search for doctors, book appointments based on real-time availability, reschedule/cancel, and view their complete medical history.
- **Doctor:** Can manage their weekly availability, view upcoming appointments, access patient history during a consultation, and complete appointments by adding diagnoses and prescriptions.
- **Admin:** Can manage (CRUD) departments, doctors, and patient profiles, and view a master list of all appointments in the hospital with status filters.
- Personalized dashboards for all three roles.
- Modern, responsive UI built with Bootstrap.

Advanced Features (Celery & Redis):

- **User-Triggered CSV Export:** Patients can request an export of their medical history. A Celery task generates the CSV in the background and emails it as an attachment, preventing UI freezing.
- **Scheduled Daily Reminders:** A Celery Beat task runs daily to send email reminders to all patients with an appointment on that day.
- **Scheduled Monthly Reports:** A Celery Beat task runs on the first of every month to generate and email an HTML activity report to every doctor.

8. Video Presentation

- **Drive Link:**
<https://drive.google.com/file/d/1a9zRvdFoWVirqJel9vWVljCWgUeRuMhm/view?usp=sharing>